VINUNIVERSITY

Spring 2025

COMP1020: Object-Oriented Programming & Data Structures

Lecture 3: Object-Oriented Design in Java I

Program Structure, Organization, Modular Programming, Class, Objects, and Exceptions.

Hieu Pham, College of Engineering & Computer Science, VinUniversity

February 25th, 2025

Course annoucement

Dear Students,

For those interested in taking the proficiency test for this course, please register using this link by **February 26, 2025 (11:59 PM).** Please note that the proficiency test will cover all topics specified in the syllabus and will have the same level of difficulty as the final exam. Below you can see some key information about the test:

- **Expected dat**e: Friday 28, 2025
- **Location**: I201, Superlab
- **Exam format:** Coding tests (practice problems)
- **Duration:** 75 minutes

The exact time will be arranged and communicated to registered students via email. If you have any questions, please feel free to reach out.

Thank you, Best regards,

The teaching team

Variables and Operators in Java (RECAP)

Last lecture, students were be able to:

- understand the expressions, variables, operators, assignments, and statements in Java.

- understand both implicit and explicit castings in Java.

Plan for today's lecture: OOP Design

Upon the completion of this lecture, students will be able to:

- understand the object-oriented design principles, including inheritance, polymorphism, encapsulation and abstraction.
- understand the program structure and organization in Java.

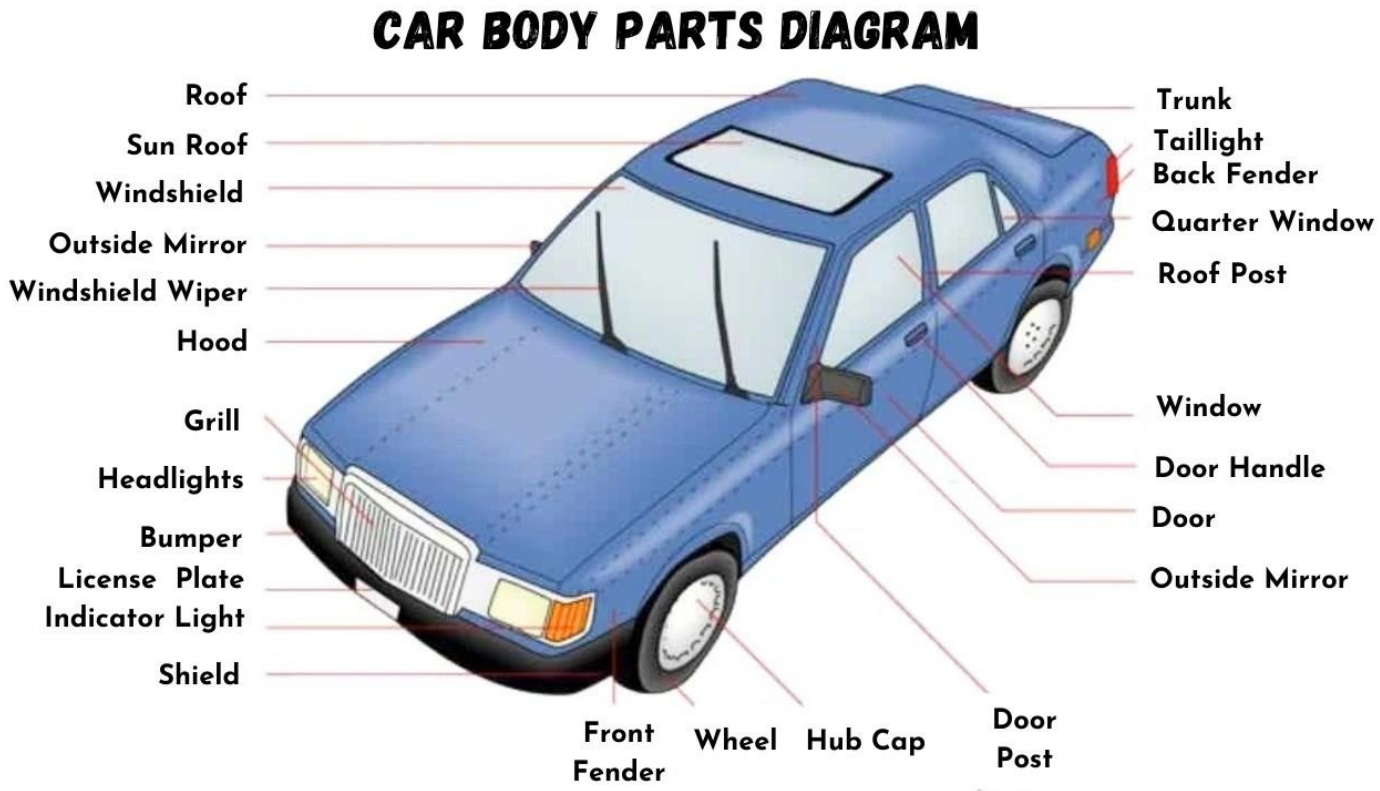
Motivation example

How to build/produce 1,000,000 cars per year?

Motivation example

How to build a car?

"Imagine building a car. You wouldn't create every component from scratch every time—you'd use pre-made parts like engines, wheels, and seats. Similarly, in programming, we use structures and modules to build complex systems efficiently."



Source

OBJECT-ORIENTED DESIGN GOALS

Three key goals:

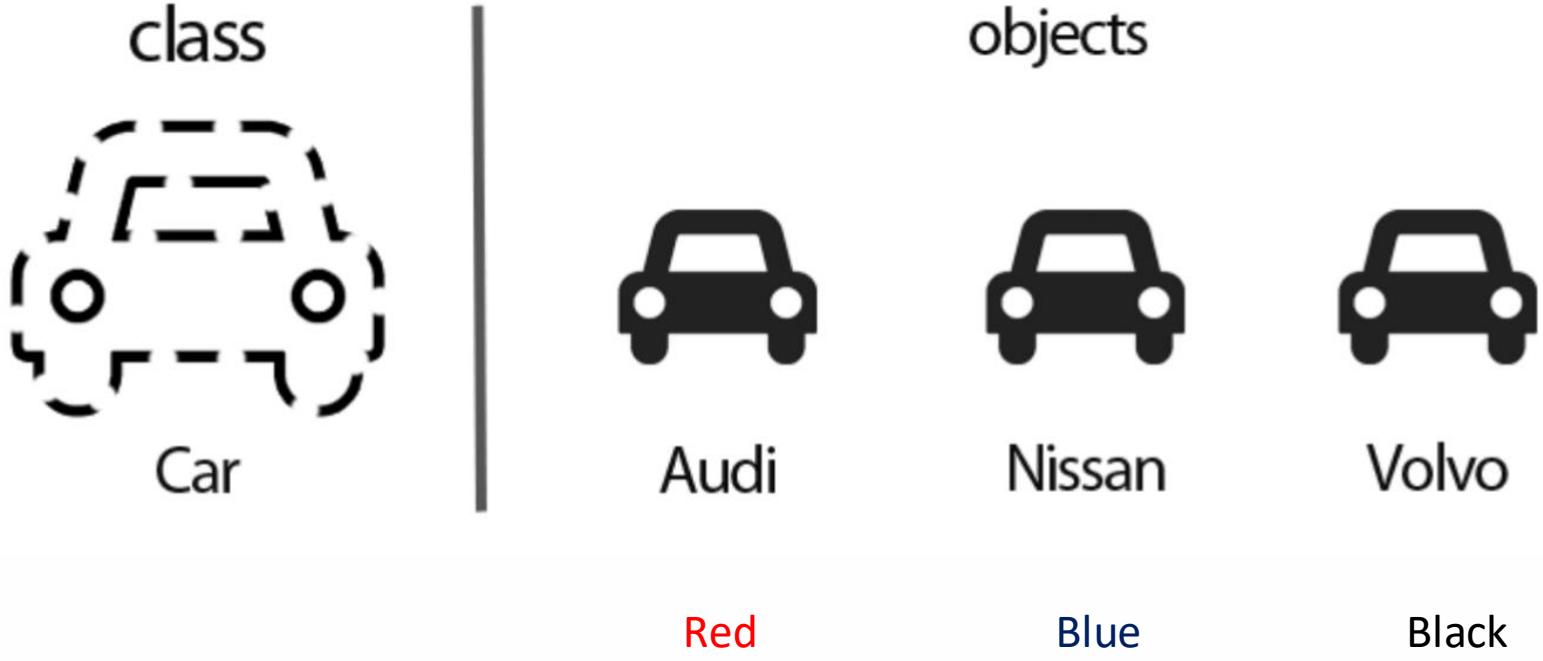
- Robustness – capable of handling unexpected inputs that are not explicitly defined for the program or application.
- Adaptability – minimal change on different hardware and platform (i.e., operating system).
- Reusability – the same code can be usable as a component of different systems in various applications.

CHARACTERISTICS OF OOP (RACAP)

- Everything is an object.
- A program is a bunch of objects telling each other what to do by sending messages.
- Every object has a type (class). All objects of a particular type can receive the same messages.
- Java is an OOP language where functionality is broken down into objects.
- An object is implemented as a Java class and has a name, some characteristics, and some actions that can be performed on it or by it.

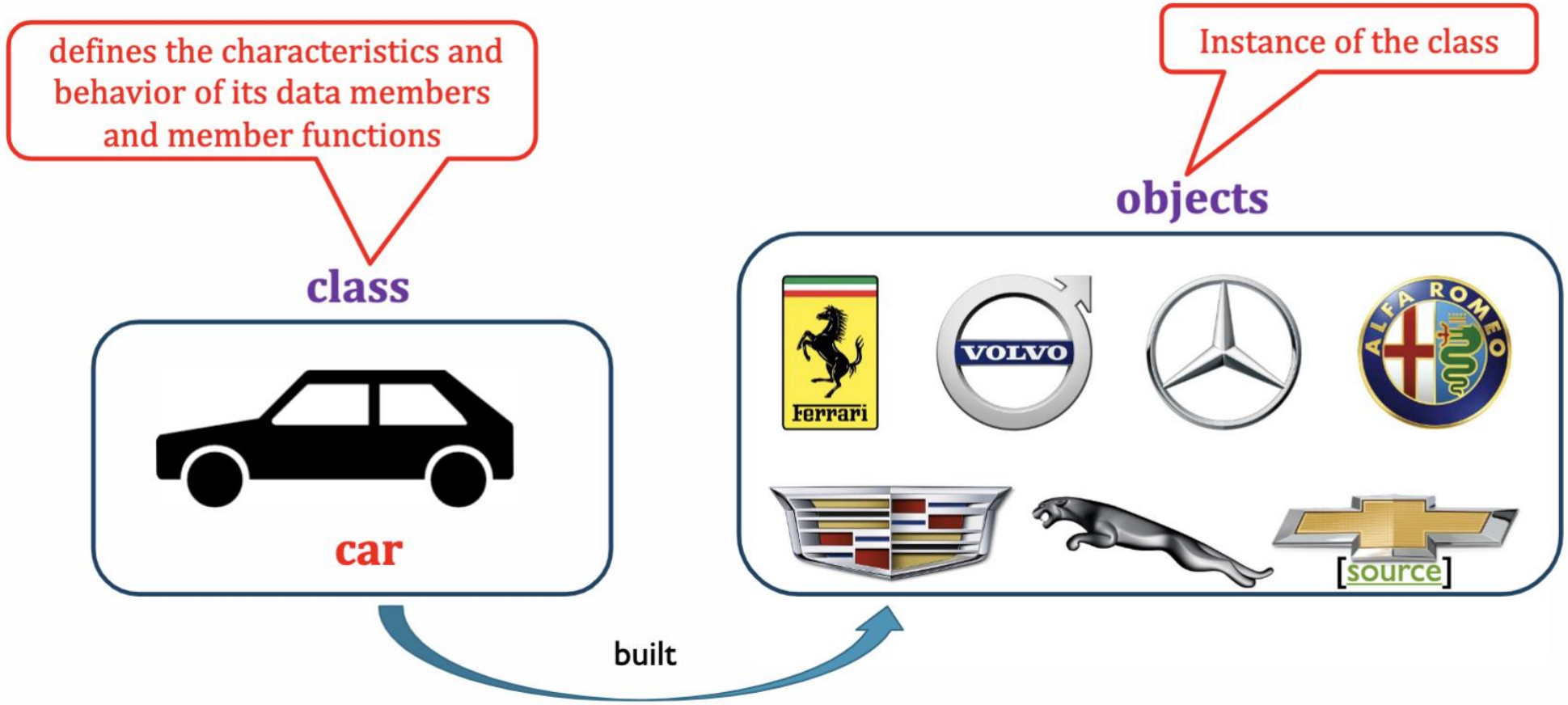
JAVA OOP

Class is a user-defined datatype that has its own data members and member functions whereas an object is an instance of class by which we can access the data members and member functions of the class.



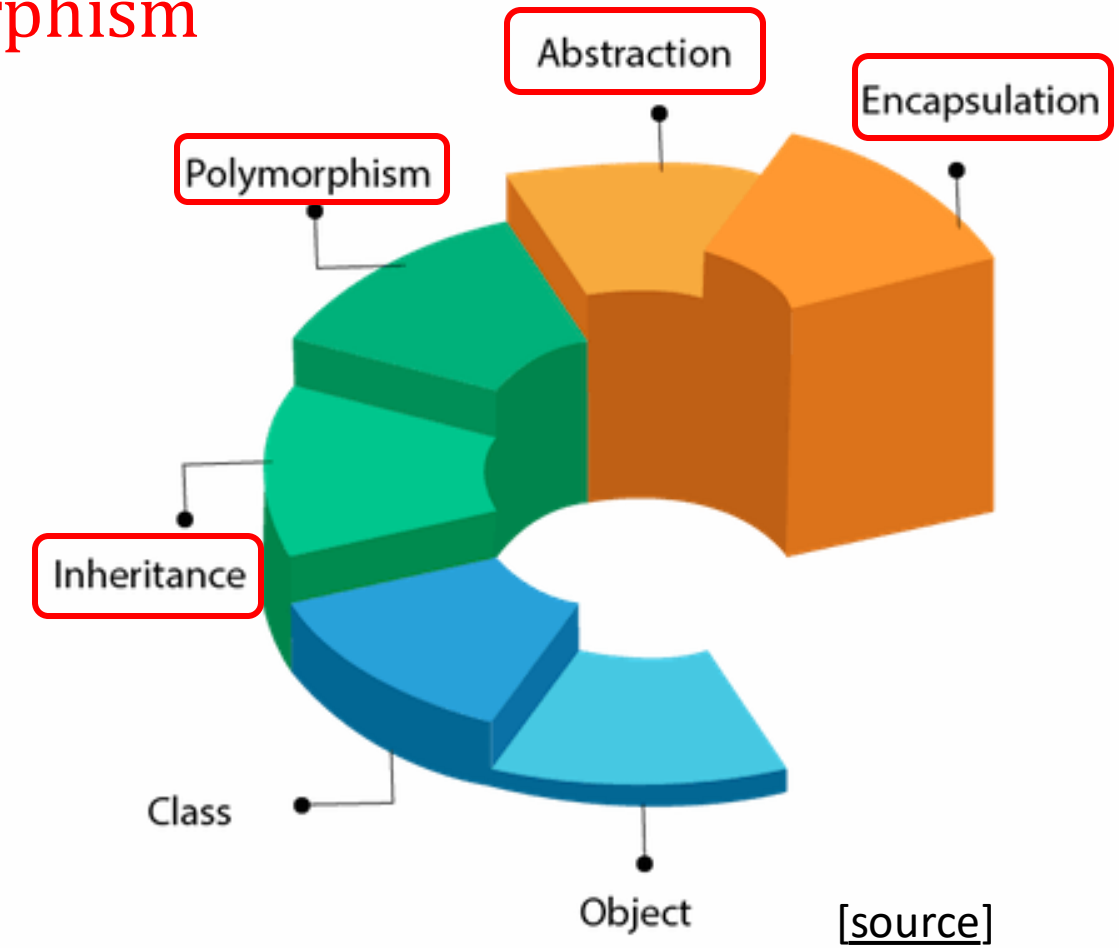
JAVA OOP

Example



OBJECT-ORIENTED DESIGN PRINCIPLES

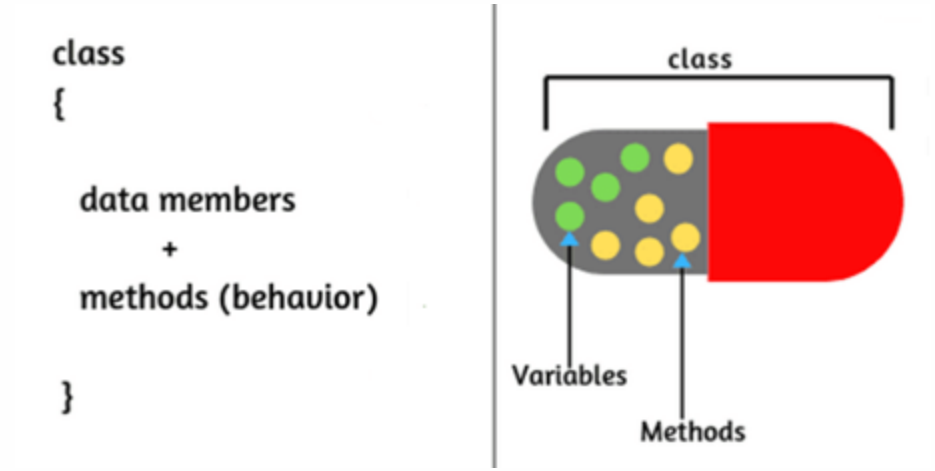
OOP is based on four principles: encapsulation, abstraction, inheritance, and polymorphism



[source]

ENCAPSULATION

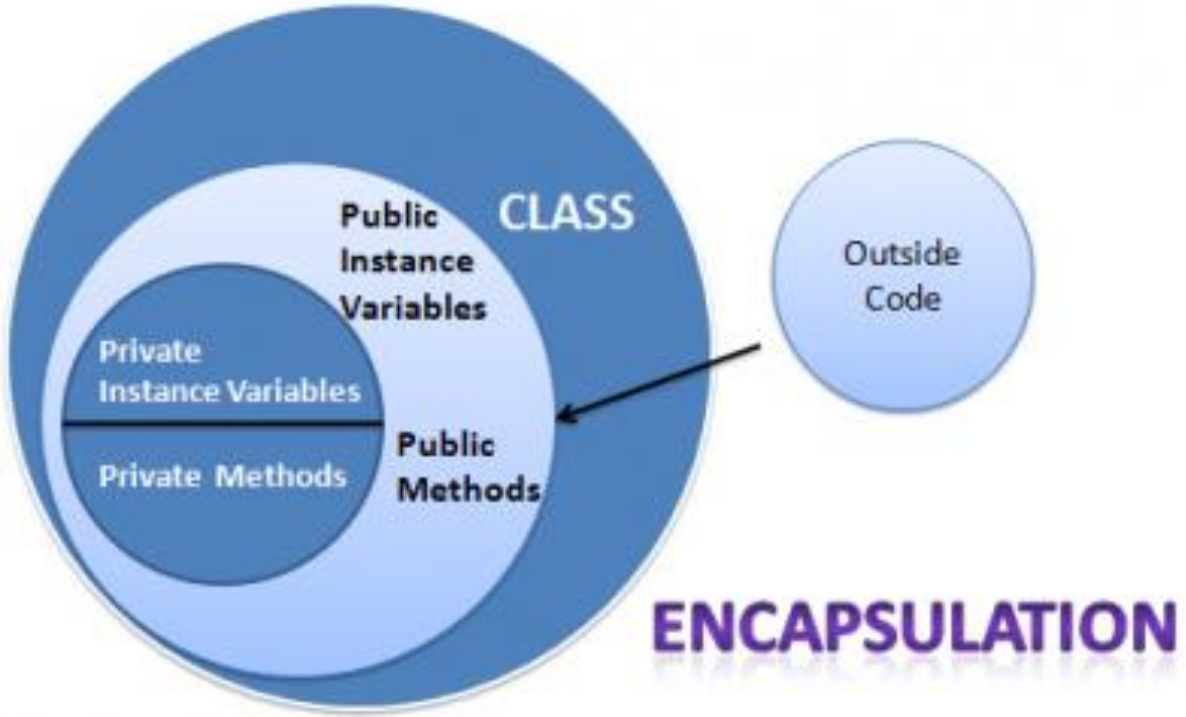
- Encapsulation: a process of binding data and functions together into a single unit such that they are kept both safe from outside interference and misuse.
- The variables of a class will be hidden from other classes and can be accessed only through the methods of their current class.



[source]

ENCAPSULATION IN JAVA

Make the instance variables private so that they cannot be accessed directly from outside the class.



[source]

ADVANTAGES OF USING ENCAPSULATION

- Flexibility – easy to change the encapsulated code with new set of requirements.
- Reusability – encapsulated code can be reused throughout the application or across multiple applications.
- Maintainability – easy to change or update application code without affecting other parts.
- Testability – easier to write unit tests since the member variables are not scattered all around.
- Data hiding – caller only knows the parameters which are to be passed to the setter method (or any method) for getting initialized with that value.

Further Reading: https://medium.com/javarevisited/why-should-encapsulation-to-be-used-e82a81f5c47c

OOP ABSTRACTION

- Abstraction: a process of hiding the implementation details from the user.
- Only the functionality will be provided to the user.
- It is used to distill a complicated system down to its most fundamental parts.
- Applying abstraction paradigm to the design of data structures gives rise to abstract data types (ADTs).

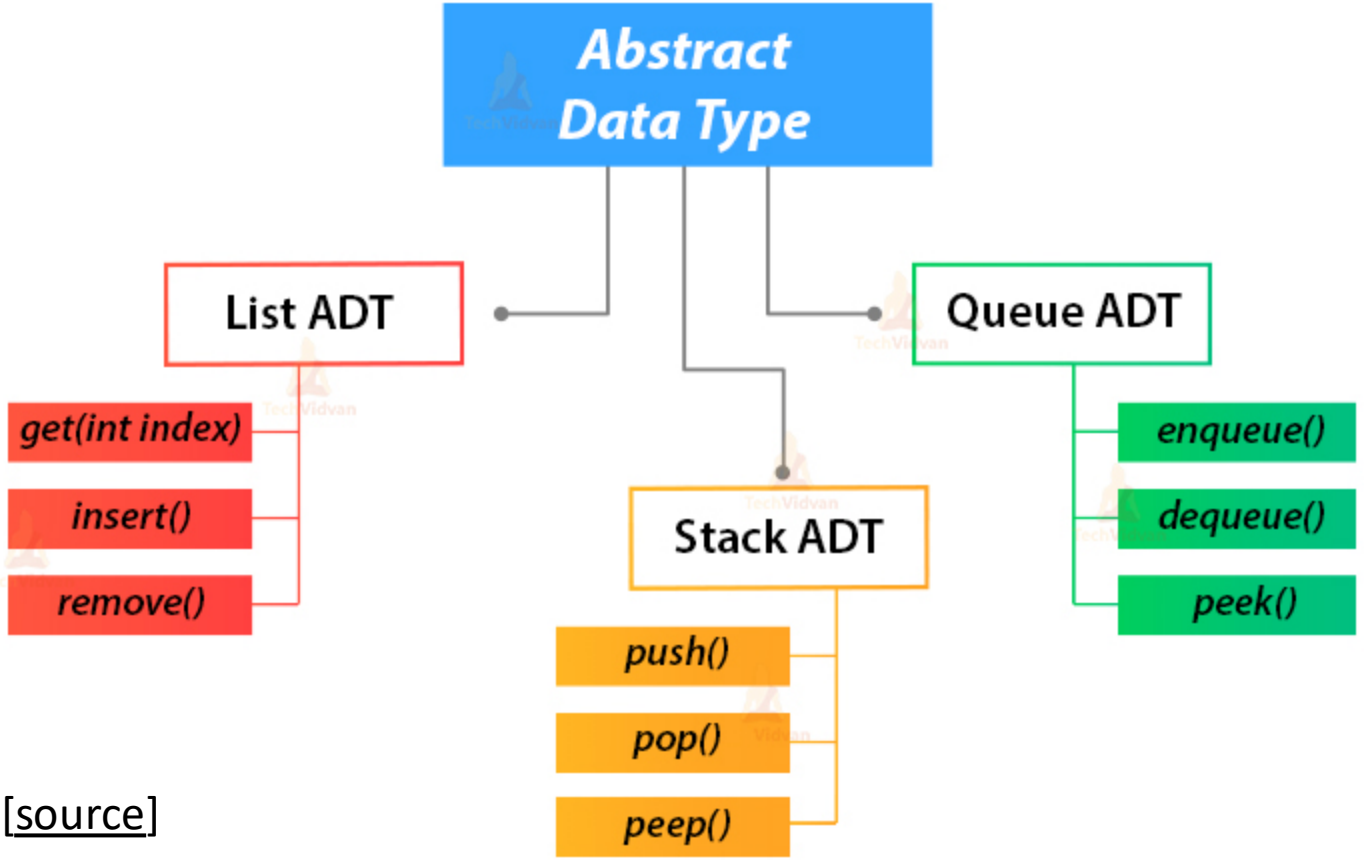
OOP ABSTRACTION

- Abstraction: a process of hiding the implementation details from the user.

Abstraction in real world. When using social platforms and contacting our friends, chatting, sharing images, etc., we don't know how these operations are happening in the background.

ABSTRACT DATA TYPES

- Abstract Data Type (ADT): a special data type that is defined by a set of values and a set of operations on that type.



[source]

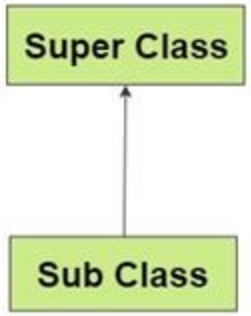
INHERITANCE

- inheritance: a process by which one class takes on the attributes and methods of another.
- super class (base class or parent class): the class that child classes are derived from.
- sub class (derived class or child class): newly formed class that can override or extend the attributes and methods of parent classes.

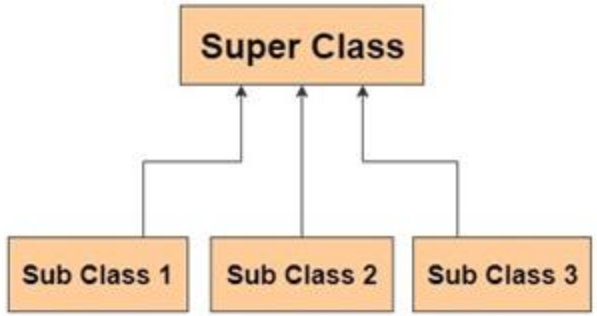
Inheritance allows us to define a class that inherits all the fields and methods from another class

TYPES OF INHERITANCE IN JAVA

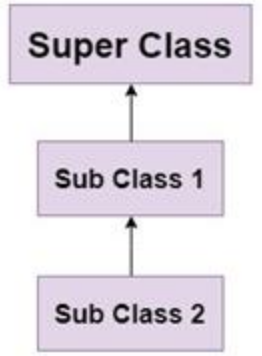
Single Inheritance



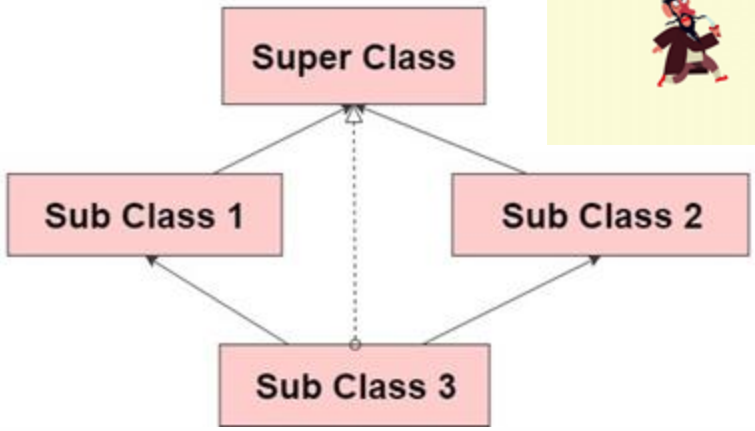
Hierarchical Inheritance

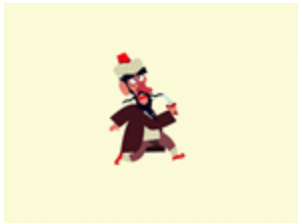


MultiLevel Inheritance

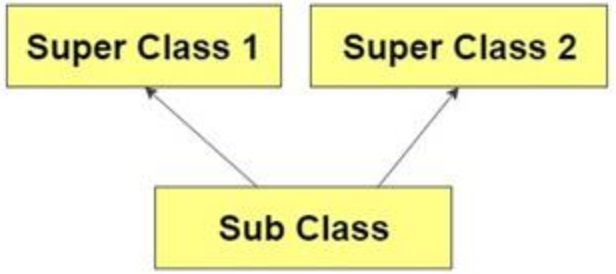


Hybrid Inheritance





Multiple Inhertance



[source]

POLYMORPHISM

- Polymorphism ("many forms"): the ability of programming languages to present the same interface for differing underlying data types.

- It refers to methods/functions/operators with the same name that can be executed on many objects or classes.

Example: A 'Car' object might have a move() method, but each subclass (ElectricCar, SportsCar) can implement its version of move()—like electric cars gliding silently while sports cars roar.

[source]

```
class Animal {
  public void animalSound() {
    System.out.println("The animal makes a sound");
  }
}

class Pig extends Animal {
  public void animalSound() {
    System.out.println("The pig says: wee wee");
  }
}

class Dog extends Animal {
  public void animalSound() {
    System.out.println("The dog says: bow wow");
  }
}

class Main {
  public static void main(String[] args) {
    Animal myAnimal = new Animal();  // Create a Animal object
    Animal myPig = new Pig();  // Create a Pig object
    Animal myDog = new Dog();  // Create a Dog object
    myAnimal.animalSound();
    myPig.animalSound();
    myDog.animalSound();
  }
}
```

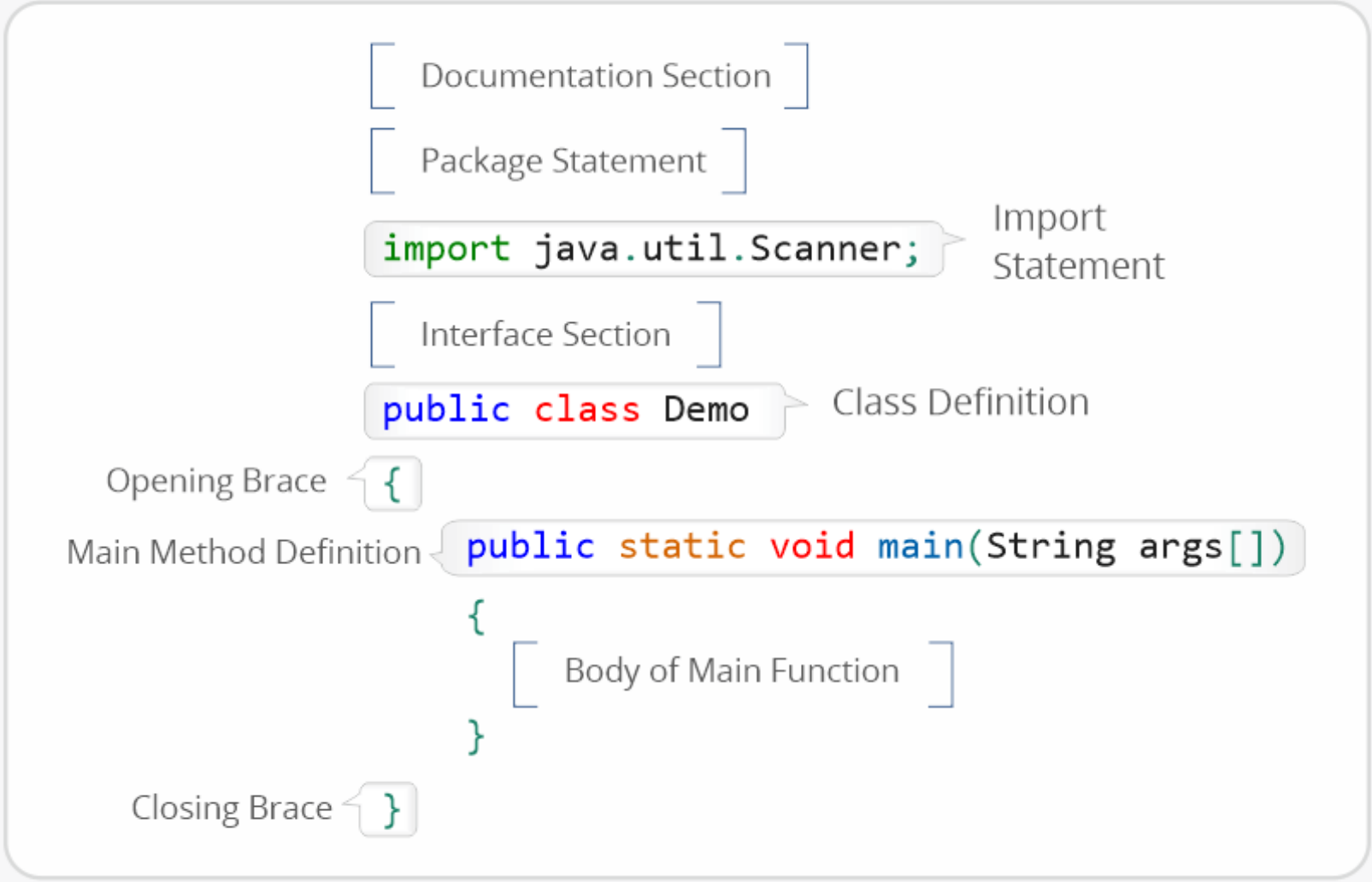
[source]

BASIC STRUCTURE OF JAVA PROGRAM

BASIC STRUCTURE OF JAVA PROGRAM

Structure of Java Program

[source]



Dremendo

DOCUMENTATION

- The documentation section includes basic information about a Java program.
- Purpose: to improve the readability of the program.
- It is an IMPORTANT section but OPTIONAL.
- Java compiler will ignore this section (i.e., statements are written as comments).

```
/**
 * Returns an Image object that ca
 * The url argument must specify a
 * argument is a specifier that is
 * <p>
 * This method always returns imme
 * image exists. When this applet
 * the screen, the data will be lo
 * that draw the image will increm
 *
 * @param  url  an absolute URL gi
 * @param  name the location of th
 * @return      the image at the s
 * @see         Image
 */
```

PACKAGE DECLARATION

- Keyword: package
- We declare the package name in which the class is placed.
- It must be defined before any class and interface declaration.
- Only one (1) package statement in a Java program.

```
package OOP; //where OOP is the package name
package COMP1020.OOP; //where COMP1020 is the root directory
                      and OOP is the subdirectory
```

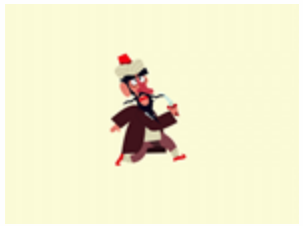
IMPORT STATEMENTS

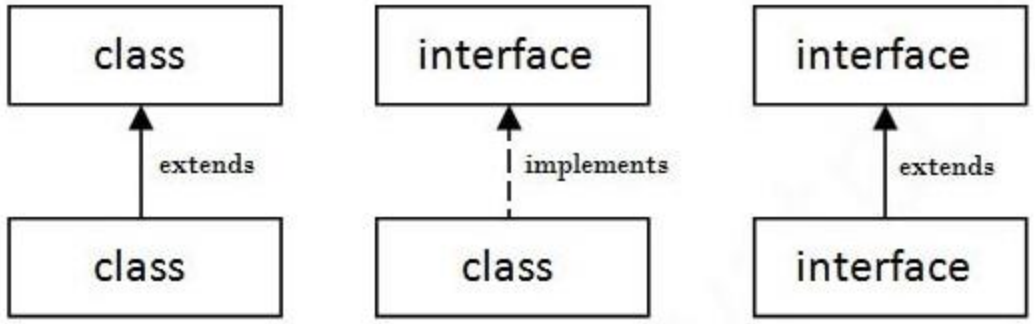
- Keyword: `import`
- In Java, if we want to use any class of a particular package, we need to import that class.
- The import statement is used to represent the class stored in the other package.

```
import java.util.Scanner; //it imports the Scanner class only
import java.util.*; //it imports all the classes of java.util package
```

INTERFACE SECTION

- Keyword: `interface`
- Interface section contains only static constants and abstract method declarations which cannot be instantiated.
- An interface is a completely abstract class that is used to gather related methods (with empty bodies) for two objects to interact.
- To access the interface methods, the interface need to be implemented by another class with the `implements` keyword.





used to achieve abstraction and multiple inheritance

INTERFACE EXAMPLE

Method with empty body

```
//Interface
interface Animal {
    public void animalSound();
    public void allCowSleep();
    int totalCow = 40;
}
```

```
//Cow "implements" the Animal interface
class Cow implements Animal {

    public void animalSound() {
        System.out.println("The cow says: moo moo");
    }

    public void allCowSleep() {
        System.out.println(totalCow + " cows are Zzz");
    }
}
```

When a class implements an interface, it must implement all of the methods declared in the interface!

```
public class MyInterface {
    public static void main(String[] args) {
        Cow myCow = new Cow();  // Create a Cow object
        myCow.animalSound();
        myCow.allCowSleep();
    }
}
```

```
The cow says: moo moo
40 cows are Zzz
```

MULTIPLE INTERFACES EXAMPLE

```java
interface Animal {
    public void animalSound();
    public void allCowSleep();
    int totalCow = 40;
}

interface ZooInfo{
    public void zooName();
    public void zooLocation();
}
```

```java
public class MyInterface {
    public static void main(String[] args) {
        Cow myCow = new Cow();
        myCow.animalSound();
        myCow.allCowSleep();
        myCow.zooName();
        myCow.zooLocation();
    }
}
```

```java
class Cow implements Animal, ZooInfo {

    public void animalSound() {
        System.out.println("The cow says: moo moo");
    }

    public void allCowSleep() {
        System.out.println(totalCow + " cows are Zzz");
    }

    public void zooName() {
        System.out.println("VinZoo");
    }

    public void zooLocation() {
        System.out.println("Ocean Park");
    }
}
```

```
The cow says: moo moo
40 cows are Zzz
VinZoo
Ocean Park
```

SELF-STUDY CORNER

You are expected to be familiar with Java control statements before the Lab 2.

Week 02 Homework

Java Control:

1. for loop/nested*
2. while-loop
3. do-while loop
4. if/if-else/if-else-if/nested*
5. switch-case
6. continue
7. break

Thank you!